NAME - Shikhar Sharma
Roll No. - 1024030898

Assignment 4

1. Problem Statement: Design a comprehensive course registration portal combining multiple collection types for optimal data handling, prerequisites tracking, and waitlist management.

Use a HashMap<String, Course> for fast course lookups by course code. Each Course object must contain a HashSet of unique enrolled student IDs, an ArrayList for ordered prerequisite codes, and a LinkedList acting as a first-come, first-served waitlist queue. Handle registration requests with capacity checks and automated waitlist promotion logic.

Answer 1

The program below uses the specified collections to register students, check prerequisites and capacity, maintain a first-come, first-served waitlist, and promote a waiting student when a seat becomes available.

```
import java.util.*;

class Course {
    String code;
    int capacity;
    HashSet<String> enrolled = new HashSet<>();
    ArrayList<String> prerequisites = new ArrayList<>();
    LinkedList<String> waitlist = new LinkedList<>();

    Course(String code, int capacity, List<String> prerequisites) {
        this.code = code;
        this.capacity = capacity;
        this.prerequisites.addAll(prerequisites);
    }
}

public class CourseRegistrationPortal {
    HashMap<String, Course> courses = new HashMap<>();
    HashMap<String, HashSet<String>> completed = new HashMap<>();

    void addCourse(String code, int capacity, List<String> prerequisites) {
        courses.put(code, new Course(code, capacity, prerequisites));
    }

    void markCompleted(String student, String course) {
        completed.computeIfAbsent(student, k -> new HashSet<>()).add(course);
    }

    boolean eligible(String student, Course course) {
        return completed.getOrDefault(student, new HashSet<>())
                        .containsAll(course.prerequisites);
    }

    String register(String student, String code) {
        Course c = courses.get(code);
        if (c == null) return "Course not found";
        if (c.enrolled.contains(student) || c.waitlist.contains(student))
            return "Already registered or waitlisted";
        if (!eligible(student, c)) return "Prerequisites not completed";

        if (c.enrolled.size() < c.capacity) {
            c.enrolled.add(student);
            return "Registration successful";
        }
        c.waitlist.addLast(student);
        return "Course full; added to waitlist";
    }
```

```
    void drop(String student, String code) {
        Course c = courses.get(code);
        if (c == null || !c.enrolled.remove(student)) return;

        while (c.enrolled.size() < c.capacity && !c.waitlist.isEmpty()) {
            String next = c.waitlist.removeFirst();
            if (eligible(next, c)) {
                c.enrolled.add(next);
                break;
            }
        }
    }
}
```

2. Problem Statement: Build a media player queue engine that supports fast insertions at arbitrary positions, recent history tracking, and bidirectional navigation.

Utilize a LinkedList<Song> to manage playback queues. Implement playlist features that allow adding songs to the front (priority queuing), removing duplicate song entries efficiently, and using List Iterator to traverse the playlist both forward and backward without index-based performance bottlenecks.

Answer 2

The program below uses a LinkedList for the playlist and recent history, adds priority songs at the front, removes duplicate song IDs, and uses ListIterator for forward and backward traversal.

```
import java.util.*;

class Song {
    String id, title;

    Song(String id, String title) {
        this.id = id;
        this.title = title;
    }

    public String toString() {
        return title;
    }
}

public class MediaPlayerQueue {
    LinkedList<Song> playlist = new LinkedList<>();
    LinkedList<Song> recentHistory = new LinkedList<>();

    void addPrioritySong(Song song) {
        playlist.addFirst(song);
    }

    void addAtPosition(int index, Song song) {
        playlist.add(index, song);
    }

    void removeDuplicates() {
        HashSet<String> seen = new HashSet<>();
        ListIterator<Song> it = playlist.listIterator();

        while (it.hasNext()) {
            Song song = it.next();
            if (!seen.add(song.id)) it.remove();
        }
    }

    void recordPlayed(Song song) {
        recentHistory.addFirst(song);
    }

    void traverseForward() {
        ListIterator<Song> it = playlist.listIterator();
        while (it.hasNext()) System.out.println(it.next());
    }

    void traverseBackward() {
        ListIterator<Song> it = playlist.listIterator(playlist.size());
        while (it.hasPrevious()) System.out.println(it.previous());
    }
}
```